Tutorial 07 para hacer en clase – Recuerde subir su enlace del repositorio antes de finalizar la clase.

Antes de iniciar:

- Recuerde terminar los tutoriales anteriores (ya que son acumulativos).
- Este tutorial muestra un ejemplo de cómo unir un proyecto "backend" en Nest.js, con uno "frontend" en Vue.js.

A.Modificación del Backend

Modificar el backend para permitir acceso del frontend:

- Ubicado en la carpeta raíz del proyecto ("backend"), modifique el archivo *src/main.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.enableCors({
    origin: 'http://localhost:5173',
  });

  app.setGlobalPrefix('api');

  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

B. Modificación del Frontend (visualización de libros)

Instalar Axios:

- Abra la terminal, y ubicado en la carpeta “frontend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm install axios
```

Desactivar Pinia:

- Ubicado en la carpeta raíz del proyecto (“frontend”), modifique el archivo *src/PiniaConfig.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { createPinia } from 'pinia';
import { watch } from 'vue';
import { bookSeeder } from '@/stores/bookseeder.js';
import { reviewSeeder } from '@/stores/reviewseeder.js';

export default class PiniaConfig {
  public static init() {
    const pinia = createPinia();

    /*const savedState = localStorage.getItem('piniaState');
    if (savedState) {
      pinia.state.value = JSON.parse(savedState);
    } else {
    ...
    watch(
      pinia.state,
      (state) => {
        localStorage.setItem('piniaState', JSON.stringify(state));
      },
      { deep: true },
    );*/

    return pinia;
```

```
  }
}
```

Borrar elementos que no vamos a utilizar por el momento:

- Ubicado en la carpeta “frontend”, elimine los siguientes archivos:
  - src/services/OtherService.ts
  - src/stores/bookstore.ts
  - src/stores/bookseeder.ts
  - src/stores/reviewstore.ts
  - src/stores/reviewseeder.ts

Modificar el BookService

- Modifique el archivo *src/services/BookService.ts* reemplace todo el código, por el nuevo código que se presenta a continuación:

Reemplazar todo el código

```
import type { BookInterface } from '@/interfaces/BookInterface';
import type { CreateBookDTO } from '@/dtos/CreateBookDTO.js';
import axios from 'axios';

export class BookService {
  private static readonly API_URL = 'http://localhost:3000/api/books';

  public static async getBooks(): Promise<BookInterface[]> {
    const { data } = await axios.get(this.API_URL);
    return data;
  }

  public static async getBookById(id: number): Promise<BookInterface> {
    const { data } = await axios.get(`${this.API_URL}/${id}`);
    return data;
  }

  public static async createBook(book: CreateBookDTO): Promise<BookInterface> {
    const { data } = await axios.post(this.API_URL, book);
    return data;
  }
}
```

Modificar el ReviewService

- Modifique el archivo *src/services/ReviewService.ts* reemplace todo el código, por el nuevo código que se presenta a continuación:

Reemplazar todo el código

```
import type { ReviewInterface } from '@/interfaces/ReviewInterface';
import axios from 'axios';

export class ReviewService {
  private static readonly API_URL = 'http://localhost:3000/api/reviews';

  static async getReviews(): Promise<ReviewInterface[]> {
    const { data } = await axios.get(this.API_URL);
    return data;
  }

  static async getReviewsByBookId(bookId: number): Promise<ReviewInterface[]> {
    const { data } = await axios.get(`${this.API_URL}/book/${bookId}`);
    return data;
  }

  static async createReview(review: Omit<ReviewInterface, 'id'>): Promise<ReviewInterface> {
    const { data } = await axios.post(this.API_URL, review);
    return data;
  }
}
```

Modificar el BooksIndexView

- Modifique el archivo *src/views/BooksIndexView.vue* reemplace todo el código, por el nuevo código que se presenta a continuación:

Reemplazar todo el código

```
<script setup lang="ts">
import { BookService } from '@/services/BookService.js';
import type { BookInterface } from '@/interfaces/BookInterface.js';
import { onMounted, ref } from 'vue';

const books = ref<BookInterface[]>([]);
```

```
onMounted(async () => {
  books.value = await BookService.getBooks();
});
</script>

<template>
  <section>
    <div class="max-w-7xl mx-auto">
      <div class="flex justify-end mb-6">
        <RouterLink
          to="/books/create"
          class="inline-block bg-blue-600 text-white font-semibold px-5 py-2 rounded hover:bg-blue-
700 transition"
          >+ Add Book</RouterLink
        >
      </div>

      <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
        <div v-for="book in books" :key="book.id">
          <div class="bg-white rounded-lg shadow-md hover:shadow-lg transition duration-300 p-6
border border-gray-200">
            <div class="flex justify-between items-center mb-2">
              <h3 class="text-xl font-semibold text-gray-800">
                {{ book.title }}
              </h3>
              <span v-if="book.stock > 0" class="bg-green-100 text-green-800 text-xs px-2 py-1
rounded-full ml-2">
                {{ book.stock }} available
              </span>
              <span v-else class="bg-red-100 text-red-800 text-xs px-2 py-1 rounded-full ml-2"> Not
available </span>
            </div>

            <div class="flex justify-center mb-4">
              <img src="https://picsum.photos/seed/picsum/536/354" alt="Book Cover" class="object-
cover rounded shadow-sm w-full h-auto" />
```

```
          </div>

          <p class="text-gray-500 text-sm mb-3">
            <i class="fas fa-tag mr-2"></i>
           {{ book.category }}
          </p>

          <div class="bg-gray-50 rounded-lg p-3 mb-4">
            <div class="flex justify-between text-sm">
              <span class="text-gray-600">Price:</span>
              <span class="font-semibold">${{ book.price }} COP</span>
            </div>
          </div>

          <div class="flex justify-center">
            <RouterLink
              :to="`/books/${book.id}`"
              class="bg-blue-100 hover:bg-blue-200 text-blue-600 font-semibold py-2 px-3 rounded
transition duration-300"
            >
              More info <i class="fas fa-info-circle"></i>
            </RouterLink>
          </div>
        </div>
      </div>
    </div>
  </div>
 </section>
</template>
```

Correr el proyecto:

- Ubicado en la carpeta “backend”, ejecute el siguiente comando:

Ejecutar en Terminal

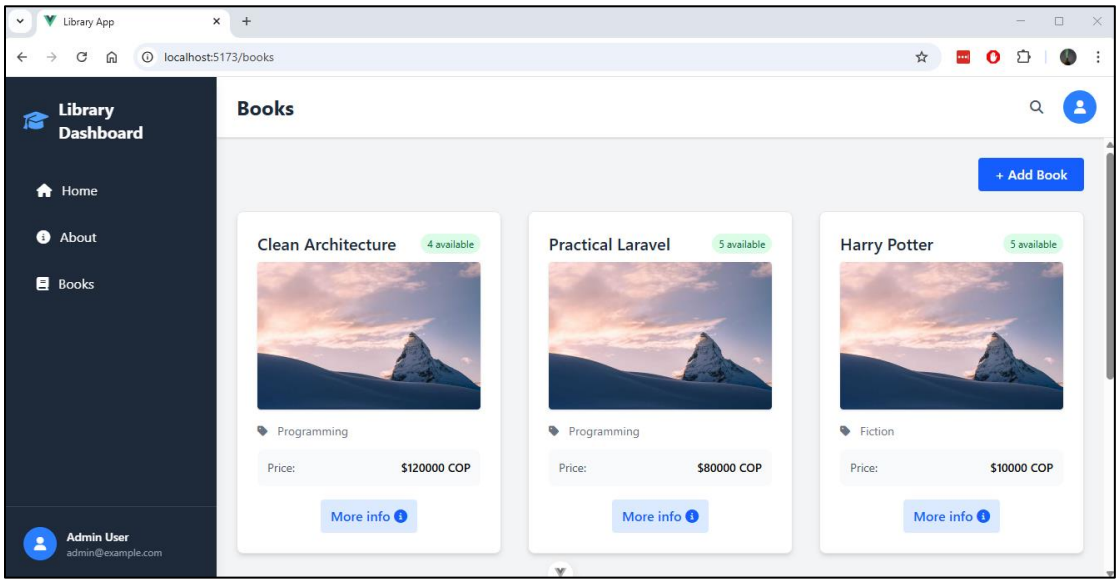
```
npm run start:dev
```

- Ubicado en la carpeta “frontend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run dev
```

- Si todo sale bien, deberá poder acceder desde el navegador a la ruta http://localhost:5173/books y ver la página de “books” que ahora está cargando libros desde el backend.



C. Modificación del Frontend (creación de libros)

Modificar BooksCreateView:

- Ubicado en la carpeta raíz del proyecto ("frontend"), modifique el archivo *src/views/BooksCreateView.vue* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
<script setup lang="ts">
...

async function submitForm() {
  const newBook: CreateBookDTO = {
    title: title.value,
    category: category.value,
    price: price.value,
    stock: stock.value,
  };
  try {
    await BookService.createBook(newBook);
    successMessage.value = 'Book created successfully!';
    title.value = '';
    category.value = '';
    price.value = 0;
    stock.value = 0;
  } catch (error) {
    console.error(error);
  }
}
</script>
```

Correr el proyecto:

- Ubicado en la carpeta "backend", ejecute el siguiente comando:

Ejecutar en Terminal

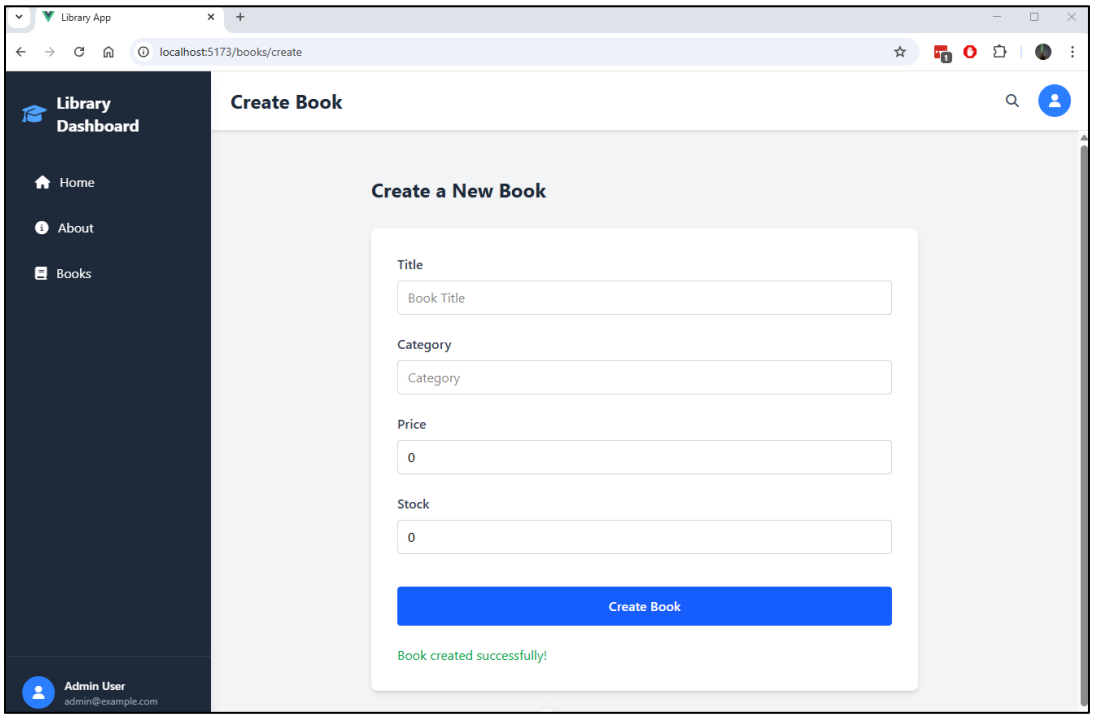
```
npm run start:dev
```

- Ubicado en la carpeta “frontend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run dev
```

- Si todo sale bien, deberá poder acceder desde el navegador a la ruta http://localhost:5173/books/create y crear libros que serán añadidos a la base de datos del backend.



D.Modificación del Backend (gestión de reseñas)

Crear controlador reviews

- Ubicado en la carpeta “backend/src/books” cree el archivo *reviews.controller.ts* con el siguiente contenido:

Añadir todo este código

```
import { Body, Controller, Get, Param, Post } from '@nestjs/common';
import { ReviewsService } from './reviews.service';
import { CreateReviewDto } from './dto/create-review.dto';
import { Review } from './entities/review.entity';

@Controller('reviews')
export class ReviewsController {
  constructor(private readonly reviewsService: ReviewsService) {}

  @Get()
  findAll(): Promise<Review[]> {
    return this.reviewsService.findAll();
  }

  @Get('book/:bookId')
  findByBookId(@Param('bookId') bookId: string): Promise<Review[]> {
    return this.reviewsService.findByBookId(Number(bookId));
  }

  @Post()
  create(@Body() createReviewDto: CreateReviewDto): Promise<Review> {
    return this.reviewsService.create(createReviewDto);
  }
}
```

Crear servicio reviews

- Ubicado en la carpeta “backend/src/books” cree el archivo *reviews.service.ts* con el siguiente contenido:

Añadir todo este código

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Review } from './entities/review.entity';
import { CreateReviewDto } from './dto/create-review.dto';

@Injectable()
export class ReviewsService {
  constructor(
    @InjectRepository(Review)
    private reviewsRepository: Repository<Review>,
  ) {}

  findAll(): Promise<Review[]> {
    return this.reviewsRepository.find();
  }

  findByBookId(bookId: number): Promise<Review[]> {
    return this.reviewsRepository.find({ where: { book: { id: bookId } } });
  }

  create(createReviewDto: CreateReviewDto): Promise<Review> {
    const { bookId, ...rest } = createReviewDto;
    const review = this.reviewsRepository.create({
      ...rest,
      book: { id: bookId },
    });

    return this.reviewsRepository.save(review);
  }
}
```

Crear entidad review

- Ubicado en la carpeta “src/books/entities” cree el archivo *review.entity.ts* con el siguiente contenido:

Añadir todo este código

```
import {
  Entity,
  Column,
  PrimaryGeneratedColumn,
  ManyToOne,
  JoinColumn,
  RelationId,
  CreateDateColumn,
} from 'typeorm';
import { Book } from './book.entity';

@Entity()
export class Review {
  @PrimaryGeneratedColumn()
  id: number;

  @ManyToOne(() => Book, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'bookId' })
  book: Book;

  @RelationId((review: Review) => review.book)
  bookId: number;

  @Column({ type: 'int' })
  rating: number;

  @Column({ type: 'text' })
  comment: string;

  @Column({ type: 'varchar', nullable: true })
  author: string | null;

  @CreateDateColumn()
```

```
  createdAt: Date;
}
```

Crear dto para la creación de reseñas

- Ubicado en la carpeta “backend/src/books/dto” cree el archivo *create-review.dto.ts* con el siguiente contenido:

Añadir todo este código

```
export class CreateReviewDto {
  bookId: number;
  rating: number;
  comment: string;
  author?: string;
}
```

Incluir reviews en el módulo de books

- Ubicado en la carpeta raíz del proyecto (“backend”), modifique el archivo *src/books/books.module.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { Module } from '@nestjs/common';
import { BooksController } from './books.controller';
import { BooksService } from './books.service';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Book } from './entities/book.entity';
import { Review } from './entities/review.entity';
import { ReviewsController } from './reviews.controller';
import { ReviewsService } from './reviews.service';

@Module({
  imports: [TypeOrmModule.forFeature([Book, Review])],
  controllers: [BooksController, ReviewsController],
  providers: [BooksService, ReviewsService],
})
export class BooksModule {}
```

Colocar la relación bidireccional en book

- Ubicado en la carpeta raíz del proyecto ("backend"), modifique el archivo *src/books/entities/book.entity.ts* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
import { Entity, Column, PrimaryGeneratedColumn, OneToMany } from 'typeorm';
import { Review } from './review.entity';

@Entity()
export class Book {
  ...

  @Column()
  stock: number;

  @OneToMany(() => Review, (review) => review.book)
  reviews: Review[];
}
```

E. Modificación del Frontend (gestión de reseñas)

Modificar el BooksReviews

- Modifique el archivo *frontend/src/components/BooksReviews.vue* reemplace todo el código, por el nuevo código que se presenta a continuación:

Reemplazar todo el código

```
<script setup lang="ts">
import { onMounted, ref } from 'vue';
import { ReviewService } from '@/services/ReviewService.js';
import type { ReviewInterface } from '@/interfaces/ReviewInterface.js';

const props = defineProps<{
  bookId: number;
}>();

const reviews = ref<ReviewInterface[]>([]);

const form = ref({
  rating: 5,
  comment: '',
  author: '',
});

const isSubmitting = ref(false);

async function submitReview() {
  if (!form.value.comment.trim()) return;
  isSubmitting.value = true;
  await ReviewService.createReview({
    bookId: props.bookId,
    rating: Math.min(5, Math.max(1, form.value.rating)),
    comment: form.value.comment.trim(),
    author: form.value.author.trim() || undefined,
  });
  form.value = { rating: 5, comment: '', author: '' };
  isSubmitting.value = false;
```

```
  getReviews();
}

function formatDate(iso?: string): string {
  if (!iso) return '';
  return new Date(iso).toLocaleDateString('es-CO', {
    year: 'numeric',
    month: 'short',
    day: 'numeric',
  });
}

async function getReviews() {
  reviews.value = await ReviewService.getReviewsByBookId(props.bookId);
}

onMounted(() => {
  getReviews();
});
</script>

<template>
  <div class="space-y-6">
    <h3 class="text-lg font-semibold text-gray-800">Reviews</h3>

    <!-- Create review form -->
    <div class="bg-gray-50 rounded-lg p-4 border border-gray-200">
      <h4 class="text-sm font-medium text-gray-700 mb-3">Add a review</h4>
      <form @submit.prevent="submitReview" class="space-y-3">
        <div>
          <label for="rating" class="block text-sm text-gray-600 mb-1">Rating</label>
          <select
            id="rating"
            v-model.number="form.rating"
            class="w-full border border-gray-300 rounded py-2 px-3 focus:outline-none focus:ring
focus:border-blue-300"
```

```
          required
        >
          <option v-for="n in 5" :key="n" :value="n">{{ n }} star{{ n > 1 ? 's' : '' }}</option>
        </select>
      </div>
      <div>
        <label for="comment" class="block text-sm text-gray-600 mb-1">Comment</label>
        <textarea
          id="comment"
          v-model="form.comment"
          rows="3"
          class="w-full border border-gray-300 rounded py-2 px-3 focus:outline-none focus:ring
focus:border-blue-300"
          placeholder="Write your review..."
          required
        />
      </div>
      <div>
        <label for="author" class="block text-sm text-gray-600 mb-1">Your name (optional)</label>
        <input
          id="author"
          v-model="form.author"
          type="text"
          class="w-full border border-gray-300 rounded py-2 px-3 focus:outline-none focus:ring
focus:border-blue-300"
          placeholder="Name"
        />
      </div>
      <button
        type="submit"
        :disabled="isSubmitting || !form.comment.trim()"
        class="bg-blue-600 text-white font-medium py-2 px-4 rounded hover:bg-blue-700
disabled:opacity-50 disabled:cursor-not-allowed transition"
      >
        Post review
      </button>
    </form>
```

```
    </div>

    <!-- Review list -->
    <ul class="space-y-4">
      <li
        v-for="review in reviews"
        :key="review.id"
        class="bg-white rounded-lg border border-gray-200 p-4 shadow-sm"
      >
        <div class="flex items-center justify-between gap-2 mb-2">
          <span class="font-medium text-gray-800">{{ review.author || 'Anonymous' }}</span>
          <span class="text-amber-500 text-sm" :title="`${review.rating} stars`">
            {{ '★'.repeat(review.rating) }}{{ '☆'.repeat(5 - review.rating) }}
          </span>
        </div>
        <p class="text-gray-600 text-sm whitespace-pre-wrap">{{ review.comment }}</p>
        <p v-if="review.createdAt" class="text-gray-400 text-xs mt-2">
          {{ formatDate(review.createdAt) }}
        </p>
      </li>
      <li v-if="reviews.length === 0" class="text-gray-500 text-sm py-4">
        No reviews yet. Be the first to review!
      </li>
    </ul>
  </div>
</template>
```

Modificar el BooksShowView

- Ubicado en la carpeta raíz del proyecto ("frontend"), modifique el archivo *src/views/BooksShowView.vue* según lo que se resalta en negrilla en el siguiente código:

Modifique según la negrilla

```
<script setup lang="ts">
...
import { useRoute } from 'vue-router';
import type { BookInterface } from '@/interfaces/BookInterface.js';
import { onMounted, ref } from 'vue';
```

```
~~const route = useRoute();~~
~~const bookId = Number(route.params.id);~~
~~const book = BookService.getBookById(bookId);~~

const book = ref<BookInterface | null>(null);

// functions
function formatToCOP(price: number): string {
  ...
}

onMounted(async () => {
  const route = useRoute();
  const bookId = Number(route.params.id);
  book.value = await BookService.getBookById(bookId);
});
</script>
...
```

Correr el proyecto:

- Ubicado en la carpeta “backend”, ejecute el siguiente comando:

Ejecutar en Terminal

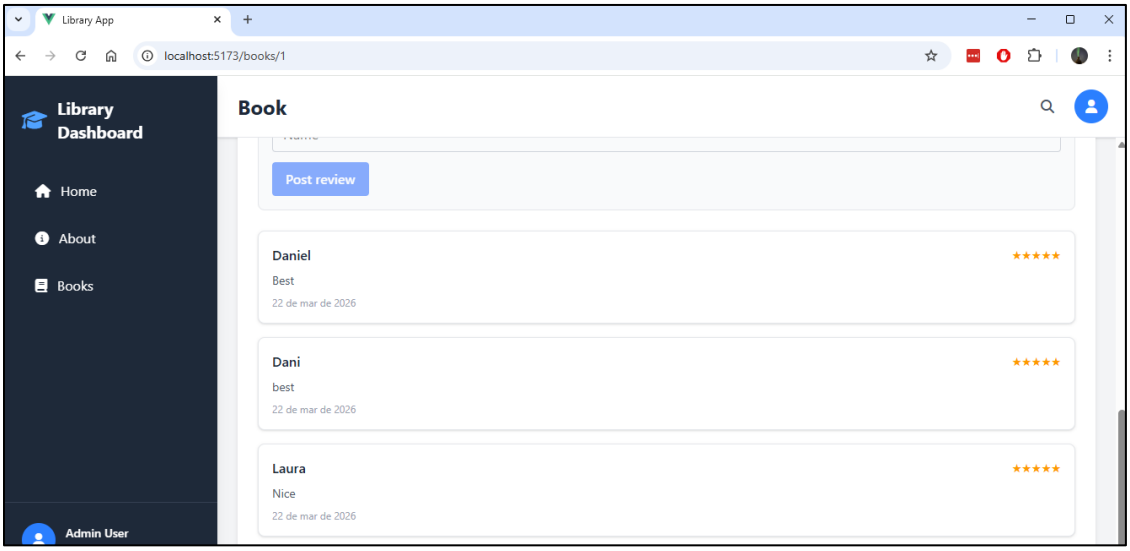
```
npm run start:dev
```

- Ubicado en la carpeta “frontend”, ejecute el siguiente comando:

Ejecutar en Terminal

```
npm run dev
```

- Si todo sale bien, deberá poder acceder desde el navegador a la ruta http://localhost:5173/books/1 y crear reseñas que serán añadidos a la base de datos del backend.



¡Felicidades! Acaba de conectar ambos proyectos y desarrollar una aplicación FullStack basada en un Backend Nest.js y un Frontend Vue.js.